OpenDMA – Open Document Management Architecture

Final
Version: 0.8
Editor: Stefan Kopf

Preface

The Open document management architecture (OpenDMA) is based on an *abstract* architecture that is able to cover all existing document management systems. This abstract architecture is defined in multiple layers (*sections* in this document), where each layer is built on top of features offered by lower layers.

Section I: Object oriented OpenDMA model

OpenDMA uses a strongly typed reflective object oriented data model with a class hierarchy. Section I.1 first defines a *simple object model* that has as few constraints as possible. Section I.2 then adds the constraints and reflection needed by OpenDMA.

Section I.1: Simple object model

This first subsection defines an object model without any constraints. Those objects are often referred to as *soft objects*.

§1 Qualified names

OpenDMA uses only qualified names. Each consists of

1. a namespace (Unicode text string with 1 or more characters), and
2. a name (noncolon Unicode text string with 1 or more characters).

The namespace is a sequence of 1 or more non-empty non-colon names separated by the colon character. Namespaces are nested from left to right by appending another non-colon name.

The namespace “opendma” is reserved for this architecture specification.

If a qualified name is represented as a simple string, the namespace and the name are concatenated by the colon character.

§2 Data types

The simple object model is able to hold data as scalar values or un-typed references to other objects. Both may either be single valued or multi valued.

§2.1 Scalar values

The OpenDMA class architecture knows these scalar data types:

1. String (Unicode)
2. Integer
3. Short integer
4. Long integer
5. Float
6. Double
7. Boolean
8. DateTime in UTC
9. Binary

§2.2 Reference values

The simple object model can reference other objects in the same context (see §4).

§2.3 Content

The special data type “Content” represents binary data as octet streams. These are typically very large in size and are accessed as a whole in a stream typed manner.

§2.4 Cardinality

Data of each data type can either be single valued or multi valued. Single valued data has at most one value of the data type whereas multi valued data is unbound and can have multiple elements of the same data type.

Multi valued data of the “Reference” data type is represented as an unordered set while multi valued data of all other data types is represented as an ordered list.

§2.5 Data type id

A numeric *data type id* is assigned to each data type corresponding to this table:

Data type	Data type id
String (§2.1)	1
Integer (§2.1)	2
Short integer (§2.1)	3
Long integer (§2.1)	4
Float (§2.1)	5

Double (§2.1)	6
Boolean (§2.1)	7
DateTime (§2.1)	8
Binary (§2.1)	9
Reference (§2.2)	10
Content (§2.3)	11

§2.6 Nullability

The OpenDMA data model knows the special value *null* as representation for “not assigned”.

Null values are only available for single valued data. Multi valued data can neither contain *null* values as individual elements nor can it be *null* in its entirety. It always contains a potentially empty list or set.

§3 Properties

A *property* is defined as a triple consisting of

1. a qualified name as defined in §1,
2. a data type and cardinality as defined in §2, and
3. a value depending on the data type or null as representation for “not assigned”.

§4 Objects

An *object* is an unordered set of zero or more properties (see §3). Each property name must be unique within the object.

Each object can be uniquely identified in its *context* by some *unique object identifier* that must be presentable as String.

A *context* is an artificial boundary set up by the actual document management system around objects in the same management domain.

§5 Operations on objects

Each object (§4) supports only these two operations:

- Read property
  input: qualified name (§1)
  output: property (§3)
  Returns the entire description of this property with its name, data type, cardinality and value(s).

- Write property
  input: qualified name (§1) and value(s)
  Modifies only the value(s) of a property identified by its name.

Both operations may succeed or fail. This result (success or failure) must be returned to the caller.

> *Notes:*
>
> - *It is not defined, when the read or write operation has to succeed and when it has to fail.*
> - *The outcome of the read and write operations is not defined. After writing a value for some property, it is not defined that a following read operation has to return this value or that it has to succeed at all.*

Section I.2: OpenDMA object model

OpenDMA enforces some constraints on the simple object model for type safety and reflection.

§6 Basic object constraints

The following constraints apply to all objects in the OpenDMA object model.

§6.1 Reflection

Every object must have at least the following two properties:

1. A single valued property (§3) with the qualified name `opendma:Class` of the "Reference" data type,
   which must contain a reference to a *valid class object* (§8.3).
2. A multi valued property (§3) with the qualified name `opendma:Aspects` of the "Reference" data type,
   which can contain an unordered set of zero or more references to *valid aspect objects* (§8.4).

The properties of every object must match exactly in number, data type, cardinality and nullability the combined *effective property lists* (§10)
defined by these valid class and aspect objects. The qualified names of each property must be unique across all effective property lists.

A reference property *x* (§2.2) must only contain references to objects whose `opendma:Class` or `opendma:Aspects` properties contain a reference to a valid
class info object that is or extends (§8.5) the class info object referenced by the `opendma:ReferenceClass` property of the property
info object (§9) describing *x*.

If a property info object has a non-empty `opendma:Choices` property, the value of the corresponding property must only contain values
described by one of the `opendma:ChoiceValue` objects.

§6.2 Identification

Every object must have at least a single valued property (§3) with the qualified name `opendma:Id` and the "String" data type. This property must contain the String representation of the unique object identifier as defined in §4.

> *Implementation notes:*
> *APIs for OpenDMA are encouraged to introduce a special data type for Ids.*

§7 Class info object

A *class info object* is an object with at least theses properties:

1. `opendma:Class`, single value, Reference, as defined in §6.1, not null
2. `opendma:Aspects`, multi value, Reference, as defined in §6.1, can be empty
3. `opendma:Id`, single value, String, as defined in §6.2, not null
4. `opendma:Name`, single value, String, not null
5. `opendma:Namespace`, single value, String, not null
6. `opendma:DisplayName`, single value, String, not null
7. `opendma:SuperClass`, single value, Reference to a class info object (§7), can be null
8. `opendma:IncludedAspects`, multi value, Reference to valid aspect objects (§8.4), can be empty
9. `opendma:DeclaredProperties`, multi value, Reference to property info objects (§9), can be empty
10. `opendma:Properties`, multi value, Reference to property info objects (§9), can be empty
11. `opendma:Aspect`, single value, Boolean, not null
12. `opendma:Hidden`, single value, Boolean, not null
13. `opendma:System`, single value, Boolean, not null
14. `opendma:Retrievable`, single value, Boolean, not null
15. `opendma:Searchable`, single value, Boolean, not null
16. `opendma:SubClasses`, multi value, Reference to a class info objects (§7), can be empty

These constraints apply to the properties of valid class objects (§8.3) and valid aspect objects (§8.4):

1. The restrictions of the `opendma:SuperClass` property are defined in §8.
2. The set of property info objects referenced by the `opendma:Properties` matches exactly the effective properties list (§10) of this class, making it effectively a derived read-only property
3. The value of the `opendma:SubClasses` property is exactly the set of valid class objects whose `opendma:SuperClass` property contains a reference to this class info object
4. The values of `opendma:IncludedAspects` are all valid aspect objects

> *Conclusion:*
> *The set of valid aspect objects of the `opendma:IncludedAspects` property contain the set of the `opendma:IncludedAspects` property of the class info object referenced by `opendma:SuperClass`*

§8 Class hierarchy

§8.1 Class hierarchy root

Every context (§4) contains at least one class info object (§7) with these property values:

Property name	Value
`opendma:Class`	Reference to the class class object (§8.2)
`opendma:Id`	Unique object identifier
`opendma:Name`	String “Object”
`opendma:Namespace`	String “opendma”
`opendma:DisplayName`	String “OdmaObject”
`opendma:SuperClass`	NULL
`opendma:DeclaredProperties`	Contains at least references to property info objects as described in §9 defining the “Class” (§6.1) and the “Id” (§6.2) properties.
`opendma:Aspect`	false

Every context has to provide a reference to exactly one object following these constraints. This object is called the *class hierarchy root*.

> *Note:*
> *Properties defined in §7 and not listed in the table above have implementation specific values. The constraints set*
> *forth in this document apply, e.g. for* `opendma:Properties`

§8.2 Class class object

Every context (§4) contains a class info object (§7) that is referenced by the `opendma:Class` property of the class hierarchy root (§8.1) with these property values:

Property name	Value
`opendma:Class`	Reference to itself
`opendma:Id`	Unique object identifier
`opendma:Name`	String “Class”
`opendma:Namespace`	String “opendma”
`opendma:DisplayName`	String “OdmaClass”
`opendma:SuperClass`	Reference to the class hierarchy root (§8.1)

`opendma:DeclaredProperties`	Contains at least references to property info objects as described in §9 defining all properties listed in §7
`opendma:Aspect`	false

This object is called the *class class*.

> *Note:*
> *Properties defined in §7 and not listed in the table above have implementation specific values. The constraints set*
> *forth in this document apply, e.g. for* `opendma:Properties`

> *Conclusion:*
> *Every context (§4) contains exactly one class class object. This follows directly from the existence and uniqueness*
> *of the class hierarchy root and the single cardinality and not nullability of the Class property of the class*
> *hierarchy root.*

§8.3 Valid class objects

A *valid class object* is a class info object (§7) following these conditions:

1. The class hierarchy root (§8.1) is a valid class object.
2. All class info objects containing a reference to a valid class object in their `opendma:SuperClass` property and whose `opendma:Class` object is an instance of the class class object (§8.2) are again valid class objects.

This forms a tree like structure called the *OpenDMA class hierarchy*. The `opendma:Aspect` property of every valid class object has to contain the value `false`.

Constraints:
The value of the tuple (`opendma:Namespace` , `opendma:Name`) must be unique across all valid class objects and valid aspect objects in a context.

> *Note:*
> *Paragraph §7 lists constraints that are only required for valid class objects.*

§8.4 Valid aspect objects

A *valid aspect object* is a class info object (§7) that meets these constraints:

1. The `opendma:Aspect` property must contain the value `true`
2. The `opendma:IncludedAspects` property must contain an empty set
3. The `opendma:SuperClass` property must either contain `null` or a reference to a valid aspect object. If not `null` , the transitive closure of `opendma:SuperClass` must be acyclic
4. The value of the tuple (`opendma:Namespace` , `opendma:Name`) must be unique across all valid class objects and valid aspect objects in a context.

5. The `opendma:Class` property must contain a reference to an object that is an instance of the class class object (§8.2)

> *Conclusion:*
> *Constraint 3 in this list guarantees that the graph created by following the* `opendma:SuperClass` *references is loop free.*

> *Note:*
> *Paragraph §7 lists constraints that are only required for valid aspect objects.*

§8.5 Extension relationship

A class info object *c* is said to *extend* a class info object *s* if and only if at least one of these conditions is met:

1. The `opendma:SuperClass` property of *c* references *s*, or
2. An entry of the `opendma:IncludedAspects` property of *c* references *s*, or
3. The class info object referenced by *c*'s `opendma:SuperClass` property extends *s*, or
4. An entry of the `opendma:IncludedAspects` property of *c* extends *s*.

Naming convention:
Properties that are declared by valid class objects that are extended by a valid class *c* are said to be "inherited by *c*".

> *Note:*
> *A class does not extend itself.*

§8.6 InstanceOf relationship

An object *o* is said to be an *instance of* a class info object *c* if the `opendma:Class` property of *o* contains a reference
to a valid class info object that is or extends *c*.

§9 Property info object

A *property info object* is an object with at least theses properties:

1. `opendma:Class`, single value, Reference, as defined in §6, not null
2. `opendma:Id`, single value, String, as defined in §6.2, not null
3. `opendma:Name`, single value, String, not null
4. `opendma:Namespace`, single value, String, not null
5. `opendma:DisplayName`, single value, String, not null
6. `opendma:DataType`, single value, Integer, not null
7. `opendma:ReferenceClass`, single value, Reference, can be null
8. `opendma:MultiValue`, single value, Boolean, not null
9. `opendma:Required`, single value, Boolean, not null
10. `opendma:ReadOnly`, single value, Boolean, not null
11. `opendma:Hidden`, single value, Boolean, not null
12. `opendma:System`, single value, Boolean, not null

13. `opendma:Choices`, multi value, Reference, can be empty

> *Conclusion:*
> *There exists exactly one valid class object (§8.3) in each context (§4) that describes property info objects.*

The qualified name of the valid class info object that describes property info objects is `opendma:PropertyInfo`.

These constraints apply to the property values of property info objects:

- The `opendma:Class` property has to contain a reference to a valid class object (§8.3) that is or extends the valid class object with the qualified name `opendma:PropertyInfo`.
- The value of `opendma:DataType` must be one of the list of numeric data type ids (§2.5).
- The value of `opendma:ReferenceClass` must contain a refence to a valid class object (§8.3) or a valid aspect object (§8.4) if and only if the value of `opendma:DataType` is `10`. It must be `null` otherwise.

§10 Effective properties list

The *effective properties list* of a valid class object (§8.3) or valid aspect object ($8.4) *c* is a list of property info objects (§9) defined as follows:

1. all property info objects (§9) of *c*'s `opendma:DeclaredProperties` property are part of the effective properties list, and
2. all property info objects (§9) of the effective properties list of the class object referenced by *c*'s `opendma:SuperClass`
   property are part of the effective properties list, unless the property is *overridden* as defined below
3. all property info objects (§9) of the effective properties list of the aspect objects referenced by *c*'s `opendma:IncludedAspects` property are part of the effective properties list.

Constraints:
The value of the tuple (`opendma:Namespace`, `opendma:Name`) must be unique across all property info objects of a valid class object or valid aspect object.

Property override:
A property declared in a super class is overridden by a property info object if all these conditions are met:

1. the `opendma:Name` and `opendma:Namespace` equals the property declared in the super class
2. the data type of both objects is `10` ("Reference")
3. the `opendma:ReferenceClass` is either narrowed down or widened, meaning that one of these conditions must be met:
    1. the valid class object referenced by `opendma:ReferenceClass` of the property info object in `opendma:DeclaredProperties` is or extends the
       valid class object referenced by `opendma:ReferenceClass` of the property info object

in the effective properties list of the
valid class object referenced by *c*'s `opendma:SuperClass` property.
2. the valid class object referenced by `opendma:ReferenceClass` of the property info object in the effective properties list of the
valid class object referenced by *c*'s `opendma:SuperClass` property is or extends the valid class object referenced by
`opendma:ReferenceClass` of the property info object in
`opendma:DeclaredProperties`

Only properties of the "Reference" data type can be overridden.

§11 Choice value object

A *choice value object* is an object with at least these properties:

1. `opendma:Class`, single value, Reference, as defined in §6, not null
2. `opendma:Id`, single value, String, as defined in §6.2, not null
3. `opendma:DisplayName`, single value, String, not null
4. `opendma:StringValue`, single value, String, nullable
5. `opendma:IntegerValue`, single value, Integer, nullable
6. `opendma:ShortValue`, single value, Short, nullable
7. `opendma:LongValue`, single value, Long, nullable
8. `opendma:FloatValue`, single value, Float, nullable
9. `opendma:DoubleValue`, single value, Double, nullable
10. `opendma:BooleanValue`, single value, Boolean, nullable
11. `opendma:DateTimeValue`, single value, DateTime, nullable
12. `opendma:BinaryValue`, single value, Binary, nullable
13. `opendma:ReferenceValue`, single value, Reference, nullable

> *Note:*
> *Due to the reflection limitations (§6.1), the `opendma:ReferenceValue` property must only contain valid references. It must*
> *only contain references to objects whose `opendma:Class` property contains a reference to a class info object that is or*
> *extends (§8.5) the class info object referenced by the `opendma:ReferenceClass` property of the property info object (§9) it*
> *is contained in.*

> *Note:*
> *Choice value objects cover only a subset of scalar value types (§2.1). For example, there are no choice values for the "Content" data type.*

§12 Failure messages

The object model knows a set of distinguished failure messages for the read / write operations (§5):

1. PropertyNotFound
2. InvalidDataType
3. ObjectNotFound
4. AccessDenied
5. ServiceFailure

§12.1 Property existence

The read and the write operation (§5) for a qualified property name *pn* on an object *o* have to return an `PropertyNotFound` (§12) response
code if and only if the effective property list (§10) of *o* does not contain a property info object that matches in its `opendma:Name` and
`opendma:Namespace` values to *pn*.

§12.2 Type safety

The write operation (§5) for a qualified property name *pn* on an object *o* has to return an `InvalidDataType` (§12) response code if and only if

1. the effective property list (§10) of *o* does contain a property info object for *pn*, and
2. one of these conditions applies:
    1. the value of `opendma:DataType` of that property info object does not match the data type of the value to be written, or
    2. the value of `opendma:MultiValue` of that property info object does not match the cardinality of the value to be written, or
    3. the value of `opendma:Required` of that property info object is `true` and the value to be written is `null` in the case of a single-valued
    property or an empty collection in the case of a multi valued property, or
    4. the value of `opendma:Choices` of that property info object is not empty and does not contain a reference to a choice value object (§11) whose
    value property corresponding to the data type contains the value to be written.

The value returned by the read operation (§5) has to be of the data type defined by the numeric data type id read from the `opendma:DataType` property
of the corresponding property info for *pn*.

§13 Core class reference

Section I.2 defines a set of properties that are required for the objects of the class hierarchy, but it does not limit the actual properties to this set.

An implementer might introduce additional properties for the class hierarchy root
`opendma:Object` without violating the conditions posed by OpenDMA.
This allows the mapping of any existing class hierarchy into the OpenDMA object model.

The meaning and expected content of the properties defined in section I.2 is documented in this paragraph.

§13.1 opendma:Object

Root of the class hierarchy. Every class in OpenDMA extends this class. All objects in OpenDMA have the properties defined for this class.

Property	Type	Card	Req/Opt	Contents
opendma:Class	Reference	Single	Required	Reference to a valid class object describing this object
opendma:Id	String	Single	Required	String representation of the unique object identifier as defined in §4

§13.2 opendma:Class

Objects of this class describe Classes and Aspects in OpenDMA. Every object in OpenDMA has a reference to an instance of this class describing it.

Property	Type	Card	Req/Opt	Contents
opendma:Class	*Reference*	*Single*	*Required*	*Reference to a valid class object describing this object*
opendma:Id	*String*	*Single*	*Required*	*String representation of the unique object identifier as defined in §4*
opendma:Name	String	Single	Required	The name part of the qualified name (§1) of this class
opendma:Namespace	String	Single	Required	The namespace part of the qualified name (§1) of this class
opendma:DisplayName	String	Single	Required	Text shown to end users to refer to this class
opendma:SuperClass	Reference	Single	Optional	Super class of this class or aspect. Value is instance of opendma:Class .
opendma:IncludedAspects	Reference	Multi	Optional	List of aspects that are included in this class. Values are instances of opendma:Class .
opendma:DeclaredProperties	Reference	Multi	Optional	List of properties declared by this class. Values are

				instances of `opendma:PropertyInfo`.
`opendma:Properties`	Reference	Multi	Optional	List of effective properties. Values are instances of `opendma:PropertyInfo`.
`opendma:Aspect`	Boolean	Single	Required	Indicates if this object represents an Aspect or a Class
`opendma:Hidden`	Boolean	Single	Required	Indicates if this class should be hidden from end users and probably administrators
`opendma:System`	Boolean	Single	Required	Indicates if instances of this class are owned and managed by the system
`opendma:Retrievable`	Boolean	Single	Required	Indicates if instances of this class can by retrieved by their Id
`opendma:Searchable`	Boolean	Single	Required	Indicates if instances of this class can be retrieved in a search
`opendma:SubClasses`	Reference	Multi	Optional	List of classes or aspects that extend this class

§13.3 opendma:PropertyInfo

Objects of this class describe properties in OpenDMA. Every object in OpenDMA has a reference to an opendma:Class which has the opendma:Properties set of PropertyInfo objects. Each describes one of the properties on the object.

Property	Type	Card	Req/Opt	Contents
`opendma:Class`	*Reference*	*Single*	*Required*	*Reference to a valid class object describing this object*
`opendma:Id`	*String*	*Single*	*Required*	*String representation of the unique object identifier as defined in §4*
`opendma:Name`	String	Single	Required	The name part of the qualified name (§1) of this property
`opendma:Namespace`	String	Single	Required	The namespace part of the qualified name (§1) of this

				property
opendma:DisplayName	String	Single	Required	Text shown to end users to refer to this property
opendma:DataType	Integer	Single	Required	Numeric data type Id
opendma:ReferenceClass	Reference	Single	Optional	The opendma:Class values of the property must be an instance of if and only if the data type is “Reference” (10), null otherwise
opendma:MultiValue	Boolean	Single	Required	Indicates if this property has single or multi cardinality
opendma:Required	Boolean	Single	Required	Indicates if at least one value is required
opendma:ReadOnly	Boolean	Single	Required	Indicates if this property can be updated
opendma:Hidden	Boolean	Single	Required	Indicates if this class should be hidden from end users and probably administrators
opendma:System	Boolean	Single	Required	Indicates if instances of this property are owned and managed by the system
opendma:Choices	Reference	Multi	Optional	List of opendma:ChoiceValue instances each describing one valid value for this property

§13.4 opendma:ChoiceValue

Objects of this class describe a possible value of a property.

Property	Type	Card	Req/Opt	Contents
opendma:Class	*Reference*	*Single*	*Required*	*Reference to a valid class object describing this object*
opendma:Id	*String*	*Single*	*Required*	*String representation of the unique object identifier as defined in §4*
opendma:DisplayName	String	Single	Required	Text shown to end users to refer to this possible value option
opendma:StringValue	String	Single	Optional	Value of the property if the data type of the property is “String”,

				null otherwise
opendma:IntegerValue	Integer	Single	Optional	Value of the property if the data type of the property is “Integer”, null otherwise
opendma:ShortValue	Short	Single	Optional	Value of the property if the data type of the property is “Short”, null otherwise
opendma:LongValue	Long	Single	Optional	Value of the property if the data type of the property is “Long”, null otherwise
opendma:FloatValue	Float	Single	Optional	Value of the property if the data type of the property is “Float”, null otherwise
opendma:DoubleValue	Double	Single	Optional	Value of the property if the data type of the property is “Double”, null otherwise
opendma:BooleanValue	Boolean	Single	Optional	Value of the property if the data type of the property is “Boolean”, null otherwise
opendma:DateTimeValue	DateTime	Single	Optional	Value of the property if the data type of the property is “DateTime”, null otherwise
opendma:BinaryValue	Binary	Single	Optional	Value of the property if the data type of the property is “Binary”, null otherwise
opendma:ReferenceValue	Reference	Single	Optional	Value of the property if the data type of the property is “Reference”, null otherwise

Section II: OpenDMA document management model

While the first section has defined a common object oriented model that could represent nearly everything, this second section defines a set of classes and aspects specific for document management.

This set is divided into two parts, a set of basic and a set of extended document management classes and aspects. The first basic set is sufficient for all fundamental document management

operations. On top of these, the second set adds extended functionality that is not required in every environment.

Section II.1: Basic document management model

The set of basic document management classes consists of:

- Repository
  A *Repository* represents a place where all Objects are stored, representing the *context* defined in §4.
  It often constitutes a data isolation boundary where objects with different management requirements or access restrictions are separated into different repositories.
  Qualified names of classes and properties as well as unique object identifiers are only unique within a repository. They can be reused across different repositories.
  Object references are limited in scope within a single repository.
  Each Repository is identified by its own unique repository identifier, representable as a text string. It allows client applications to tell different repositories
  apart when working with multiple repositories at once.

- Document
  A *Document* is the atomic element users work on in a content based environment. It can be compared to a file in a file system. Unlike files, it may consist of multiple
  octet streams. These content streams can for example contain images of the individual pages that make up the document. A Document is able to keep track of its changes (versioning) and manage the access to it (checkin and checkout).

- ContentElement
  A *ContentElement* represents one atomic content element the Documents are made of. This base class defines the type of content and the
  position of this element in the sequence of all content elements.

- DataContentElement
  A *DataContentElement* represents one atomic octet stream. The binary data is stored together with meta data like size and filename.

- ReferenceContentElement
  A *ReferenceContentElement* represents a reference to external data. The reference is stored as URI to the content location.

- VersionCollection
  A *VersionCollection* represents the set of all versions of a Document. Based on the actual document management system, it can represent a single series of versions,
  a tree of versions, or any other versioning concept.

- Container
  A *Container* holds a set of containable objects that are said to be contained in this Container. This set of containees can be built up with Association objects based

on references to the container and the containee. This allows an object to be contained in multiple Containers or in no Container at all. A Container does not enforce a loop-free single rooted tree. Use a folder instead for this requirement.

- Containable
  The *Containable* aspect is used by all classes and aspects that can be contained in a Container.
- Folder
  A *Folder* is an extension of the Container forming one single rooted loop-free tree.
- Association
  An *Association* represents the directed link between a Container and a Containable object.

§14 Repository class

The `opendma:Repository` class extends the `opendma:Object` class and declares these additional properties:

Property	Type	Card	Req/Opt	Contents
`opendma:Name`	String	Single	Required	The internal technical name of this repository
`opendma:DisplayName`	String	Single	Required	Text shown to end users to refer to this repository
`opendma:RootClass`	Reference	Single	Required	Valid class object describing the class hierarchy root
`opendma:RootAspects`	Reference	Multi	Optional	Set of valid aspect objects without a super class
`opendma:RootFolder`	Reference	Single	Optional	Object that has the `opendma:Folder` aspect representing the single root if this repository has a dedicated folder tree, null otherwise

Objects of this class describe a Repository. As `opendma:Repository` inherits from `opendma:Object`, it has an `opendma:Id` property.
It contains the unique object identifier of this repository descriptor object within its repository. This Id is different from
the Id of the repository.

§14.1 Repository reflection in the objects

The `opendma:Object` class is extended as follows to reflect the Repository an object belongs to:

Property	Type	Card	Req/Opt	Contents
opendma:Repository	Reference	Single	Required	Object that is an instance of opendma:Repository describing the repository this object belongs to

§15 Global unique identification

The opendma:Object class is extended as follows:

Property	Type	Card	Req/Opt	Contents
opendma:Guid	String	Single	Required	Global unique identifier for this object as a combination of the Id of the Repository and the Id of the object.

> *Note:*
> *The Guid is a stable external identifier only, with no meaning within the scope of the repository. It is intended to help applications*
> *that wish to work with objects from multiple repositories.*

> *Implementation notes:*
> *APIs for OpenDMA are encouraged to introduce a special data type for Guids.*

§16 AuditStamped aspect

The opendma:AuditStamped aspect declares these properties:

Property	Type	Card	Req/Opt	Contents
opendma:CreatedAt	DateTime	Single	Optional	Timestamp when this object has been created
opendma:CreatedBy	String	Single	Optional	User who created this object
opendma:LastModifiedAt	DateTime	Single	Optional	Timestamp when object has been modified the last time
opendma:LastModifiedBy	String	Single	Optional	User who modified object the last time

§17 Document aspect

The opendma:Document aspect declares these properties:

Property	Type	Card	Req/Opt	Contents
opendma:Title	String	Single	Optional	The title of this document

opendma:Version	String	Single	Optional	Identifier of this version consisting of a set of numbers separated by a dot (e.g. 1.2.3)
opendma:VersionCollection	Reference	Single	Optional	Reference to the collection of all versions or null if versioning is not supported. Reference class: opendma:VersionCollection
opendma:VersionIndependentId	String	Single	Required	Id identifying this logical document independent from the specific version
opendma:VersionIndependentGuid	String	Single	Required	Guid identifying this logical document independent from the specific version
opendma:ContentElements	Reference	Multi	Optional	References to multiple ContentElement objects. Reference class: opendma:ContentElement
opendma:CombinedContentType	String	Single	Optional	The combined content type of the whole Document, calculated from the content types of each ContentElement.
opendma:PrimaryContentElement	Reference	Single	Optional	The dedicated primary ContentElement. May only be null if ContentElements is empty. Reference class: opendma:ContentElement
opendma:CheckedOut	Boolean	Single	Required	Indicates if this document is checked out
opendma:CheckedOutAt	DateTime	Single	Optional	Timestamp when this version of the document has been checked out, null if this document is not checked out
opendma:CheckedOutBy	String	Single	Optional	User who checked out this version of this document,

				null if this document is not checked out

§18 ContentElement aspect

The `opendma:ContentElement` aspect declares these properties:

Property	Type	Card	Req/Opt	Contents
`opendma:ContentType`	String	Single	Optional	The content type (aka MIME type) of the content represented by this element
`opendma:Position`	Integer	Single	Optional	The position of this element in the list of all content elements of the containing document

§19 DataContentElement aspect

The `opendma:DataContentElement` aspect extends `opendma:ContentElement` and declares these additional properties:

Property	Type	Card	Req/Opt	Contents
`opendma:Content`	Content	Single	Optional	The binary data of this content element
`opendma:Size`	Long	Single	Optional	The size of the data in number of octets
`opendma:FileName`	String	Single	Optional	The optional file name of the data

§20 ReferenceContentElement aspect

The `opendma:ReferenceContentElement` aspect extends `opendma:ContentElement` and declares these additional properties:

Property	Type	Card	Req/Opt	Contents
`opendma:Location`	String	Single	Optional	The URI where the content is stored

§21 VersionCollection aspect

The `opendma:VersionCollection` aspect declares these properties:

Property	Type	Card	Req/Opt	Contents
`opendma:Versions`	Reference	Multi	Required	Set of all versions of a document. Reference class: `opendma:Document`

`opendma:Latest`	Reference	Single	Optional	Latest version of a document. Reference class: `opendma:Document`
`opendma:Released`	Reference	Single	Optional	Latest released version of a document if a version has been released. Reference class: `opendma:Document`
`opendma:InProgress`	Reference	Single	Optional	Latest checked out working copy of a document. Reference class: `opendma:Document`

§22 Container aspect

The `opendma:Container` aspect declares these properties:

Property	Type	Card	Req/Opt	Contents
`opendma:Title`	String	Single	Optional	The title of this container
`opendma:Containees`	Reference	Multi	Optional	Set of containable objects contained in this container. Reference class: `opendma:Containable`
`opendma:Associations`	Reference	Multi	Optional	Set of associations between this container and the contained objects. Reference class: `opendma:Association`

§23 Folder aspect

The `opendma:Folder` aspect extends `opendma:Container` and declares these additional properties:

Property	Type	Card	Req/Opt	Contents
`opendma:Parent`	Reference	Single	Optional	The parent folder that directly contains this folder. Reference class: `opendma:Folder`.
`opendma:SubFolders`	Reference	Multi	Optional	The set of folders that are considered subfolders of this folder. Reference class: `opendma:Folder`.

The `opendma:Parent` property must not be `null`, except for the folder referenced in the `opendma:RootFolder` property of the Repository (§14).

The `opendma:Parent` property of the folder referenced in the `opendma:RootFolder` property of the Repository (§14) must be `null`.

The transitive closure of all `opendma:Parent` relationships must form a loop-free, single-rooted tree.

Folders that reference this folder in their `opendma:Parent` property should appear in its `opendma:SubFolders` set.
The `opendma:SubFolders` set may also contain additional folders that do not reference this folder in their `opendma:Parent` property.
These additional entries are used to represent secondary or alternative parent relationships.
The directed graph formed by all `opendma:SubFolders` links must be acyclic.
No folder may be reachable from itself by following one or more `opendma:SubFolders` references.

If the underlying ECM system represents folder relationships as a directed acyclic graph (multiple parents, but loop-free),
the OpenDMA adaptor must select one of these parents to expose as the single `opendma:Parent`.
Many systems provide a notion of a “primary parent” to support this selection.
The adaptor may expose other parent relationships of the underlying system as additional entries in the `opendma:SubFolders` sets
of the respective parent folders, subject to the acyclicity requirement.

It is undefined whether the folders referenced in the `opendma:SubFolders` set are also contained in the `opendma:Containees` property.
It is also undefined whether there are corresponding association objects in `opendma:Associations` for each object in the `opendma:SubFolders` set.

§24 Containable aspect

The `opendma:Containable` aspect declares these properties:

Property	Type	Card	Req/Opt	Contents
`opendma:ContainedIn`	Reference	Multi	Optional	Set of container objects this Containable is contained in. Reference class: `opendma:Container`
`opendma:ContainedInAssociations`	Reference	Multi	Optional	Set of associations that bind this Containable in the Container objects. Reference class: `opendma:Association`

§25 Association aspect

The `opendma:Association` aspect declares these properties:

Property	Type	Card	Req/Opt	Contents
`opendma:Name`	String	Single	Required	The name of this association
`opendma:Container`	Reference	Single	Required	Source of this directed link. Reference class: `opendma:Container`
`opendma:Containable`	Reference	Single	Required	Destination of this directed link. Reference class: `opendma:Containable`

Section II.2: Extended document management model

Version 0.8 of this standard does not define document management data models beyond the basic model.

The following features are currently discussed to be added to this standard:

- Renditions
- Annotations or notes
- Taxonomies like tags and categories
- Compound documents
- Legel holds
- Retention and Disposition
- Permissions and access control

Copyright and Licensing